

Getting Torque, RPM & Power to Read

A step-by-step fix for the Conv_Simple_1.9 PLC program

The short version

Your GS10 drive is working perfectly. On its keypad it shows real torque (73.7%) and speed (880 rpm at 30 Hz). The problem is only that the PLC is not grabbing those three numbers (torque, rpm, power) from the drive - so they show up as 0 on the dashboard and in the logger.

This guide fixes the PLC so it captures them. It is a 10-15 minute change in Connected Components Workbench (CCW): adjust one variable, paste in a corrected block of code, Build, and Download.

Why it is broken (in plain terms)

The drive holds all its live numbers in a list of 'registers'. The PLC reads them over the RS-485 cable. The current program reads them by TAKING TURNS:

- Turns 1 & 2: the basic readings (frequency, current, DC bus, fault).
- Turn 3: torque + rpm, OR power.

The trouble: it reuses ONE reader and keeps switching what it asks for every turn. The basic readings get most of the turns, so they work. But the torque/rpm/power turn keeps getting dropped before the answer is stored - so those three values never update and just sit at 0.

The fix (in plain terms)

Stop taking turns. Instead, read the WHOLE block of drive numbers in ONE go. A single read grabs all 16 status values at once - including torque, rpm and power. It is still just one message per cycle on the RS-485 bus (no extra traffic), and it removes the fragile turn-taking completely.

Step by step in CCW

1. Open the Conv_Simple_1.9 project in Connected Components Workbench.
2. Check the 'read_data' variable. In the Global Variables list, find read_data - it is the buffer that holds the drive's readings. Its Dimension must be at least 1..16 (room for 16 values). If it shows something smaller (e.g. 1..10), change the Dimension to 1..16. Leave its type as it is (WORD). NOTE: read_data is SUPPOSED to be an array - this is the one variable where a Dimension is correct (unlike the scalar vfd_* tags, which must stay blank).
3. Open the Prog_init program. Find the read section - the block near the '500 ms tick' comment that contains 'read_sel', 'mb_read_status', and the lines that set ElementCnt to 7 / 2 / 1. That whole block is what we replace.
4. Select that entire read block and replace it with the corrected code on the next page (it is also already in the repo file plc/Prog_init_ConvSimple_v1.9.st). Everything BELOW it (the motor control / write part) stays unchanged.
5. Build. It should compile with no errors. (read_sel and lp_toggle become unused - that is fine; you can leave them declared.)
6. Download to the PLC.
7. Tell Claude to re-run the live logger. Torque, rpm and power should now read real values that track the keypad (about 73% torque, ~880 rpm at 30 Hz).

The corrected code to paste

This replaces the old 'read_sel' turn-taking block. It reads drive registers 0x2100 through 0x210F in one shot and stores every value, including torque (slot 12), rpm (slot 13) and power (slot 16).

```
(* --- 500 ms tick; alternate read/write on the shared Ch2 --- *)
poll_timer(IN := NOT poll_timer.Q, PT := T#500ms);

IF poll_timer.Q THEN
    vfd_poll_count := vfd_poll_count + 1;
    poll_phase := NOT poll_phase;      (* read tick <-> write tick *)
END_IF;

(* --- ONE read per read-tick: full status block 0x2100..0x210F (16 regs) --- *)
read_local_cfg.Channel      := 2;      (* embedded RS-485 -- NOT 0 *)
read_local_cfg.TriggerType := 0;
read_local_cfg.Cmd         := 3;      (* FC03 read holding *)
read_local_cfg.ElementCnt  := 16;     (* 0x2100..0x210F = 16 regs *)
read_target_cfg.Node       := 1;
read_target_cfg.Addr       := 16#2101; (* Addr = wire+1 -> wire 0x2100 *)

mb_read_status(IN          := poll_timer.Q AND poll_phase,
               LocalCfg    := read_local_cfg,
               TargetCfg   := read_target_cfg,
               LocalAddr   := read_data);

IF mb_read_status.Q AND NOT mb_read_status.Error THEN
    vfd_fault_code := read_data[1];    (* 0x2100 fault      *)
    vfd_status_word := read_data[2];   (* 0x2101 status     *)
    vfd_frequency  := read_data[4];    (* 0x2103 Hz x100    *)
    vfd_current    := read_data[5];    (* 0x2104 A x100     *)
    vfd_dc_bus     := read_data[6];    (* 0x2105 V x10      *)
    vfd_voltage    := read_data[7];    (* 0x2106 V x10      *)
    vfd_torque     := read_data[12];   (* 0x210B torque % x10 *)
    vfd_motor_rpm  := read_data[13];   (* 0x210C motor rpm  *)
    vfd_power      := read_data[16];   (* 0x210F power kW x1000 *)
    IF vfd_fault_code > 0 THEN
        vfd_last_fault := vfd_fault_code; (* latch trip cause *)
    END_IF;
    vfd_comm_ok := TRUE;
END_IF;
IF mb_read_status.Error THEN
    vfd_comm_ok := FALSE;
    vfd_err_count := vfd_err_count + 1;
END_IF;
```

How to know it worked

Success:

After Download, the logger shows torque/rpm/power moving with the motor and matching the keypad. rpm should be about (Hz x 120 / poles) minus a little slip - for example ~880 rpm at 30 Hz on a 4-pole motor.

If they are STILL 0:

Then the drive is refusing to return that block over RS-485 (an addressing/comms issue, not this code). Tell Claude - the next step would be to confirm the exact register addresses the drive answers to. But since the keypad shows the values, the single-read fix is very likely all that is needed.

What this does NOT change

- The motor control (forward / reverse / stop) - untouched.
- The frequency command on the dashboard - still comes from vfd_freq_sp.
- The drive's own settings - nothing on the GS10 keypad changes.